

Working with Forms & User Input

Easy Guide

Contents

1	What Are Forms & What's Tricky About Them?	2
1.1	Understanding Forms	2
1.1.1	Common Form Elements	2
1.2	Why Are Forms Tricky?	2
2	Handling Form Submission	3
2.1	What Happens When User Submits?	3
2.2	Basic Steps	3
3	Managing & Getting User Input via State	4
3.1	State - The Core Concept	4
3.1.1	How State Works	4
3.2	Generic Handler Pattern	4
4	Getting User Input via Refs	5
4.1	What Are Refs?	5
4.1.1	When to Use Refs	5
4.2	Refs vs State	5
5	Getting Values via FormData & Native Browser APIs	6
5.1	FormData API	6
5.1.1	Benefits	6
5.2	How It Works	6
5.3	Native Browser APIs	6
6	Resetting Forms	7
6.1	Why Reset Forms?	7
6.2	How to Reset	7
6.2.1	Reset via State	7
6.2.2	Reset via Ref	7
7	Validating Input on Every Keystroke	8
7.1	Real-Time Validation	8
7.2	How It Works	8
7.3	Common Validations	8

8	Validating Input Upon Lost Focus (Blur)	9
8.1	What is Blur?	9
8.2	Why Use Blur Validation?	9
8.3	How It Works	9
9	Validating Input Upon Form Submission	10
9.1	Final Validation Check	10
9.2	Steps	10
9.3	Common Checks	10
10	Validating Input via Built-in Validation Props	11
10.1	HTML5 Validation	11
10.2	Common Attributes	11
10.3	Benefits	11
11	Mixing Custom & Built-in Validation Logic	12
11.1	Best Approach	12
11.2	How to Combine	12
11.3	Example Situation	12
12	Building & Using a Reusable Input Component	13
12.1	Why Reusable Components?	13
12.2	Benefits	13
12.3	What Should Be in Component?	13
12.4	How to Use	13
13	Outsourcing Validation Logic	15
13.1	Separation of Concerns	15
13.2	How to Do It	15
13.3	Benefits	15
13.3.1	Example	15
14	Creating a Custom useInput Hook	16
14.1	What is a Custom Hook?	16
14.2	Why useInput Hook?	16
14.3	What It Does	16
14.3.1	Benefits	16
15	Using Third-Party Form Libraries	17
15.1	Popular Form Libraries	17
15.2	Why Use a Library?	17
15.3	When to Use?	17
15.4	When to Skip?	17

Chapter 1

What Are Forms & What's Tricky About Them?

1.1 Understanding Forms

Forms are one of the most important parts of web applications. They help users send information to your application. Think of a form like a paper questionnaire - users fill it out and send it to you.

1.1.1 Common Form Elements

- Text input boxes
- Dropdown menus
- Checkboxes
- Radio buttons
- Text areas
- Buttons (Submit, Reset)

1.2 Why Are Forms Tricky?

Forms are challenging because you need to:

1. **Collect data** - Get what the user types
2. **Track changes** - Know when the user types something new
3. **Validate input** - Check if the data is correct
4. **Handle errors** - Show helpful error messages
5. **Submit data** - Send the information to the server
6. **Manage state** - Remember what the user entered

Key Challenge: Keeping track of what the user enters and updating the screen in real-time can be complicated!

Chapter 2

Handling Form Submission

2.1 What Happens When User Submits?

When a user clicks the submit button, your form should:

1. Stop the default page reload (prevent losing data)
2. Get all the form data
3. Validate the information
4. Send it to the server
5. Show success or error message

2.2 Basic Steps

1. Listen for the “submit” event
2. Use `event.preventDefault()` to stop page reload
3. Process the form data
4. Send to backend or process locally

Tip: Always prevent the default form submission behavior first!

Chapter 3

Managing & Getting User Input via State

3.1 State - The Core Concept

State is like the “memory” of your form. It remembers what the user typed in each field.

3.1.1 How State Works

- User types something
- Your code updates the state
- The screen refreshes to show the new state
- This happens many times per second

3.2 Generic Handler Pattern

Use a single handler function for multiple inputs:

```
function handleChange(event) {  
  const { name, value } = event.target;  
  setFormData({ ...formData, [name]: value });  
}
```

This approach:

- Reduces code repetition
- Makes it easier to add new fields
- Keeps your component clean

Best Practice: Use generic handlers instead of creating a unique handler for each field!

Chapter 4

Getting User Input via Refs

4.1 What Are Refs?

Refs let you access HTML elements directly, without using state. Think of it as getting information from the element when you need it.

4.1.1 When to Use Refs

- Getting file upload inputs
- Focusing on an input field
- Triggering animations
- Integrating with third-party libraries

4.2 Refs vs State

Feature	State	Refs
Tracks changes automatically	Yes	No
Re-renders when changed	Yes	No
Good for forms	Yes	Sometimes
Direct element access	No	Yes

Remember: Use state for most form situations. Use refs only when you need direct access to the element.

Chapter 5

Getting Values via FormData & Native Browser APIs

5.1 FormData API

FormData is a browser tool that automatically collects all form data without you writing much code.

5.1.1 Benefits

- Handles file uploads easily
- Works with all input types
- Less code to write
- Native browser feature

5.2 How It Works

1. Get the form element
2. Create FormData from the form
3. Send it to the server
4. No need to manually track each field

5.3 Native Browser APIs

Other useful browser APIs:

- `HTMLFormElement` - Access form methods
- `HTMLInputElement` - Access input properties
- `ValidityState` - Check input validity

Chapter 6

Resetting Forms

6.1 Why Reset Forms?

After successful submission, you usually want to clear all the fields so the form is ready for new data.

6.2 How to Reset

1. Set state back to initial empty values
2. Or use the form's built-in `reset()` method
3. Or use a ref to call `reset()` on the form element

6.2.1 Reset via State

```
setFormData({ name: '', email: '', message: '' });
```

6.2.2 Reset via Ref

```
formRef.current.reset();
```

Best Practice: Always reset after successful submission!

Chapter 7

Validating Input on Every Keystroke

7.1 Real-Time Validation

Check if the user's input is correct as they type. This gives instant feedback.

7.2 How It Works

1. User types a character
2. Your handler function runs
3. Check if input is valid
4. Show error message if needed
5. Update the screen

7.3 Common Validations

- Email format (must have @)
- Password strength (must be 8+ characters)
- Name field (no empty values)
- Phone number (correct format)

Warning: Real-time validation can be slow if checking takes time. Be careful!

Chapter 8

Validating Input Upon Lost Focus (Blur)

8.1 What is Blur?

Blur happens when the user clicks away from an input field. It's a good time to validate because the user is done typing in that field.

8.2 Why Use Blur Validation?

- Less frequent than keystroke validation
- User won't see errors while still typing
- Better performance
- Better user experience

8.3 How It Works

1. User types in a field
2. User clicks away (blur event)
3. Your validation runs
4. Show error if needed

Tip: Blur validation is often the best balance between catching errors and not annoying the user!

Chapter 9

Validating Input Upon Form Submission

9.1 Final Validation Check

This is your last chance to check everything before sending data to the server.

9.2 Steps

1. User clicks submit
2. Check all fields at once
3. If any field is invalid, show all errors
4. Don't send data to server
5. If all fields valid, send the data

9.3 Common Checks

- Required fields are not empty
- Email is in correct format
- Password meets requirements
- Passwords match
- User agrees to terms

Best Practice: Always validate on submission, even if you validated earlier!

Chapter 10

Validating Input via Built-in Validation Props

10.1 HTML5 Validation

HTML5 gives you built-in validation attributes you can use without JavaScript.

10.2 Common Attributes

Attribute	What It Does
<code>required</code>	Field cannot be empty
<code>type="email"</code>	Must be valid email
<code>minlength="8"</code>	Minimum 8 characters
<code>maxlength="20"</code>	Maximum 20 characters
<code>pattern="[0-9]"</code>	Must match pattern

10.3 Benefits

- Simple to use
- Browser handles validation
- Works without JavaScript
- Mobile keyboards change based on type

Remember: Always validate on the server too! Users can disable JavaScript.

Chapter 11

Mixing Custom & Built-in Validation Logic

11.1 Best Approach

Use both HTML5 attributes AND custom JavaScript validation together.

11.2 How to Combine

1. Use HTML5 for basic checks (`required`, `type="email"`)
2. Use JavaScript for complex checks
3. Show errors from both types

11.3 Example Situation

- HTML5: Check email format
- JavaScript: Check if email already exists

Smart Strategy: Use HTML5 for simple validation, custom code for complex logic!

Chapter 12

Building & Using a Reusable Input Component

12.1 Why Reusable Components?

Instead of writing the same form input code many times, create one component and use it everywhere.

12.2 Benefits

- Less code to write
- Easier to maintain
- Consistent styling
- Consistent behavior
- Easy to fix bugs

12.3 What Should Be in Component?

- The input element
- Label
- Error message display
- Styling
- Validation logic

12.4 How to Use

Instead of:

```
<input type="email" onChange={handleChange} />  
<error-message>{errors.email}</error-message>
```

Use:

```
<Input type="email" onChange={handleChange}  
      error={errors.email} />
```

Pro Tip: Build a component library for your project!

Chapter 13

Outsourcing Validation Logic

13.1 Separation of Concerns

Keep validation logic separate from your component code. This makes everything cleaner and easier to test.

13.2 How to Do It

1. Create a separate file for validation functions
2. Write functions like `validateEmail()`, `validatePassword()`
3. Import and use these functions
4. Your component just calls the functions

13.3 Benefits

- Easier to test validation
- Can use same validation in multiple places
- Cleaner component code
- Easy to update validation rules

13.3.1 Example

File: `validation.js`

```
export function validateEmail(email) {  
  return email.includes('@');  
}  
  
export function validatePassword(password) {  
  return password.length >= 8;  
}
```

Best Practice: Always separate validation logic from your components!

Chapter 14

Creating a Custom useInput Hook

14.1 What is a Custom Hook?

A custom hook is a JavaScript function that uses other hooks. It packages common logic so you can reuse it.

14.2 Why useInput Hook?

A custom hook for inputs helps you:

- Manage input state easily
- Handle changes automatically
- Reset fields quickly
- Reuse in many components

14.3 What It Does

1. Manages the input value in state
2. Provides a handler for changes
3. Provides a reset function
4. Returns everything you need

14.3.1 Benefits

- Less code in your components
- Consistent behavior everywhere
- Easy to add new features
- Easy to test

Smart Approach: Create custom hooks for repeated patterns!

Chapter 15

Using Third-Party Form Libraries

15.1 Popular Form Libraries

Several libraries make form handling much easier:

- **Formik** - Easy form management
- **React Hook Form** - Lightweight and fast
- **React Final Form** - Powerful and flexible

15.2 Why Use a Library?

- Less code to write
- Better performance
- Built-in validation
- Easy error handling
- Already tested

15.3 When to Use?

- Complex forms with many fields
- Need advanced validation
- Need cross-field validation
- Building a large application

15.4 When to Skip?

- Simple forms (2-3 fields)
- Learning React

- Small project
- Need full control

Decision: Start simple, use a library when forms get complex!

Quick Summary

Key Takeaways

1. Forms need careful handling of user input
2. Use state to track form data
3. Validate at multiple points (keystroke, blur, submit)
4. Use HTML5 validation for simple checks
5. Create reusable components
6. Keep validation logic separate
7. Use custom hooks to avoid repetition
8. Consider libraries for complex forms

Golden Rules

1. **Always validate on the server** - Never trust client-side validation alone
2. **Reset after success** - Clear the form after successful submission
3. **Show helpful errors** - Tell users what went wrong and how to fix it
4. **Keep it simple** - Don't overcomplicate your forms
5. **Test your forms** - Make sure they work for all users

Remember: Good forms make for happy users!